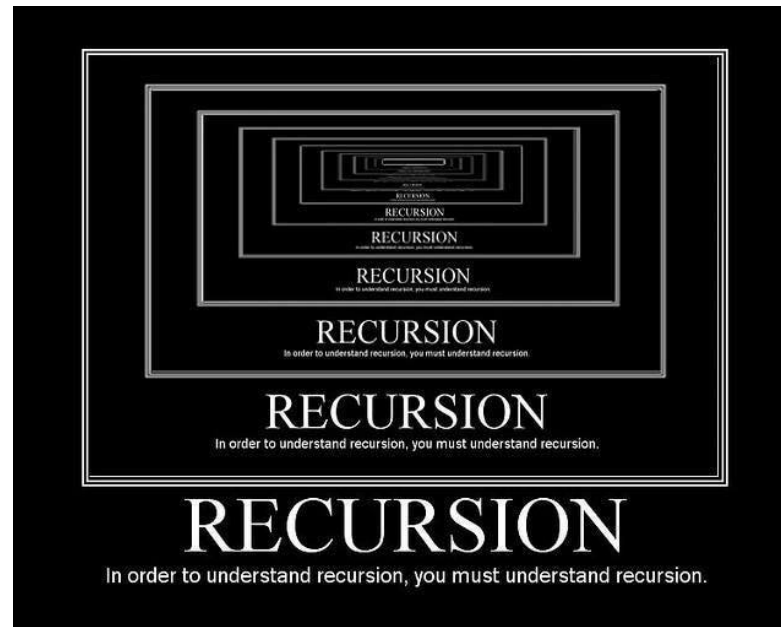


# Rekursion



# Rekursion



Rekursion (lat. “zurücklaufen”) tritt auf, wenn eine Methode sich **selbst direkt** oder **indirekt aufruft**.

Jede rekursive Methode muss eine **erreichbare Abbruchbedingung** haben, um eine Endlos-Rekursion zu vermeiden (ansonsten Stackoverflow).

Rekursion wird oft verwendet, um Probleme zu lösen, die sich in kleinere, **ähnliche Teilprobleme zerlegen** ("Teile und Herrsche") lassen.

Diese Teilprobleme lassen sich anschließend gut parallelisiert berechnen.

# Rekursion: Beispiel 1/2

```
class MathUtil {  
  
    static int faculty(int n) {  
        // Abort condition  
        if (n == 0) {  
            return 1;  
        }  
        // Recursive loop  
        return n * faculty(n - 1);  
    }  
  
    void main() {  
        int n = 3;  
        int result = faculty(n);  
        IO.printf("%d! = %d", n, result); // Output: 3! = 6  
    }  
}
```



"Die Fakultät ist in der Mathematik diejenige Funktion, die jeder natürlichen Zahl das Produkt aller positiven natürlichen Zahlen zuordnet, die diese Zahl nicht übertreffen. Sie wird durch ein dem Funktionsargument nachgestelltes Ausrufezeichen („!") abgekürzt." – Wikipedia

# Rekursion: Beispiel 2/2

```
class MathUtil {
```

```
    static int faculty(int n) {
```

```
        // Abort condition
```

```
        if (n == 0) {  
            return 1;  
        }
```

```
        // Recursive loop
```

```
        return n * faculty(n - 1); // n * faculty(n); würde zur Endlosrekursion führen.
```

```
    }
```

```
void main() {
```

```
    int n = 3;
```

```
    int result = faculty(n);
```

```
    IO.printf("%d! = %d", n, result); // Output: 3! = 6
```

```
}
```

```
}
```

Damit die Abbruchbedingung in diesem Beispiel erreichbar ist, darf der Parameter **nicht konstant** sein!

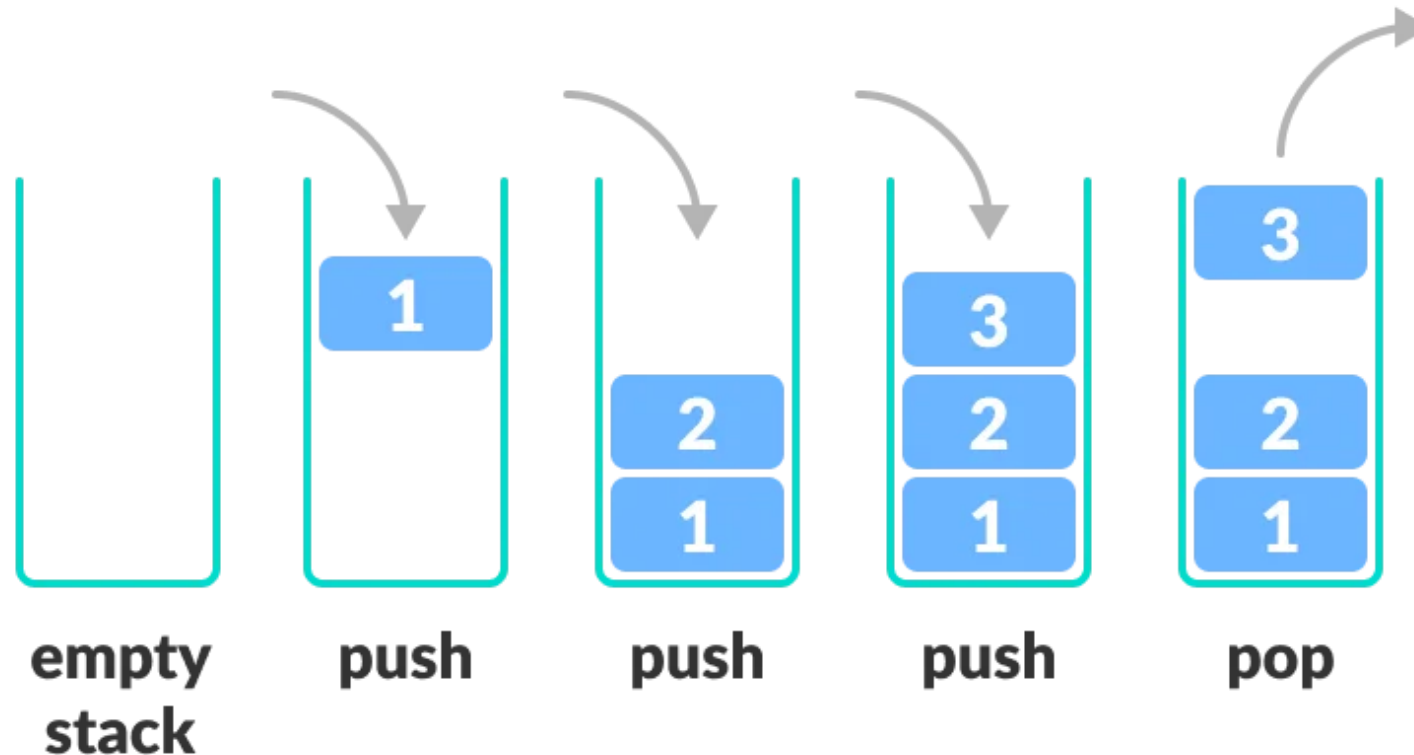


"Die Fakultät ist in der Mathematik diejenige Funktion, die jeder natürlichen Zahl das Produkt aller positiven natürlichen Zahlen zuordnet, die diese Zahl nicht übertreffen. Sie wird durch ein dem Funktionsargument nachgestelltes Ausrufezeichen („!") abgekürzt." – Wikipedia



# Stack (Deque)

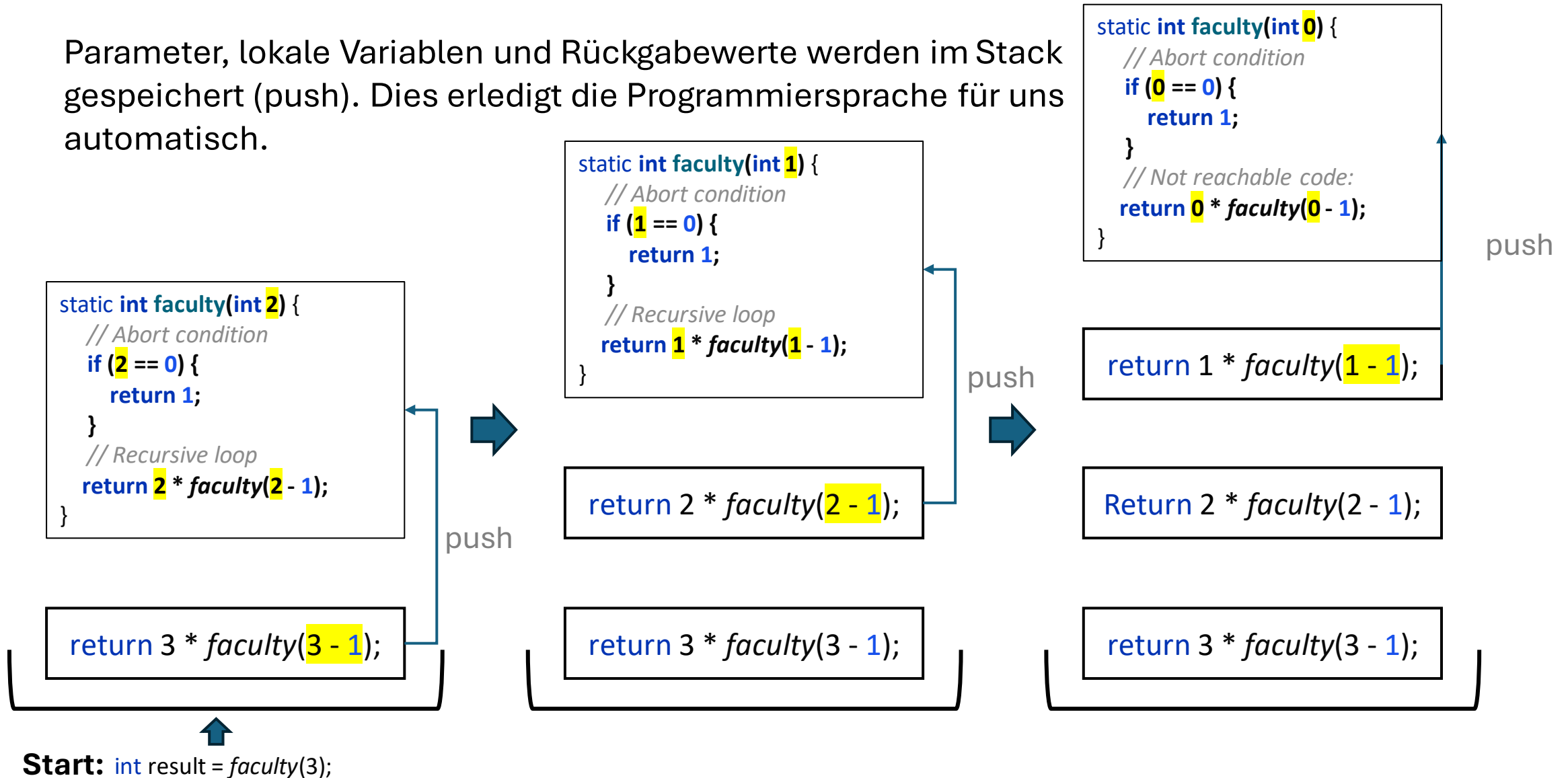
Ein **Stack** (in Java: **Deque** bzw. **LinkedList**) ist eine **lineare Datenstruktur**, bei der ein neues Element immer auf das zuletzt hinzugefügte Element kommt (**push**) oder das zuletzt hinzugefügte Element entfernt wird (**pop**).





# Stack: Rekursiver Aufbau

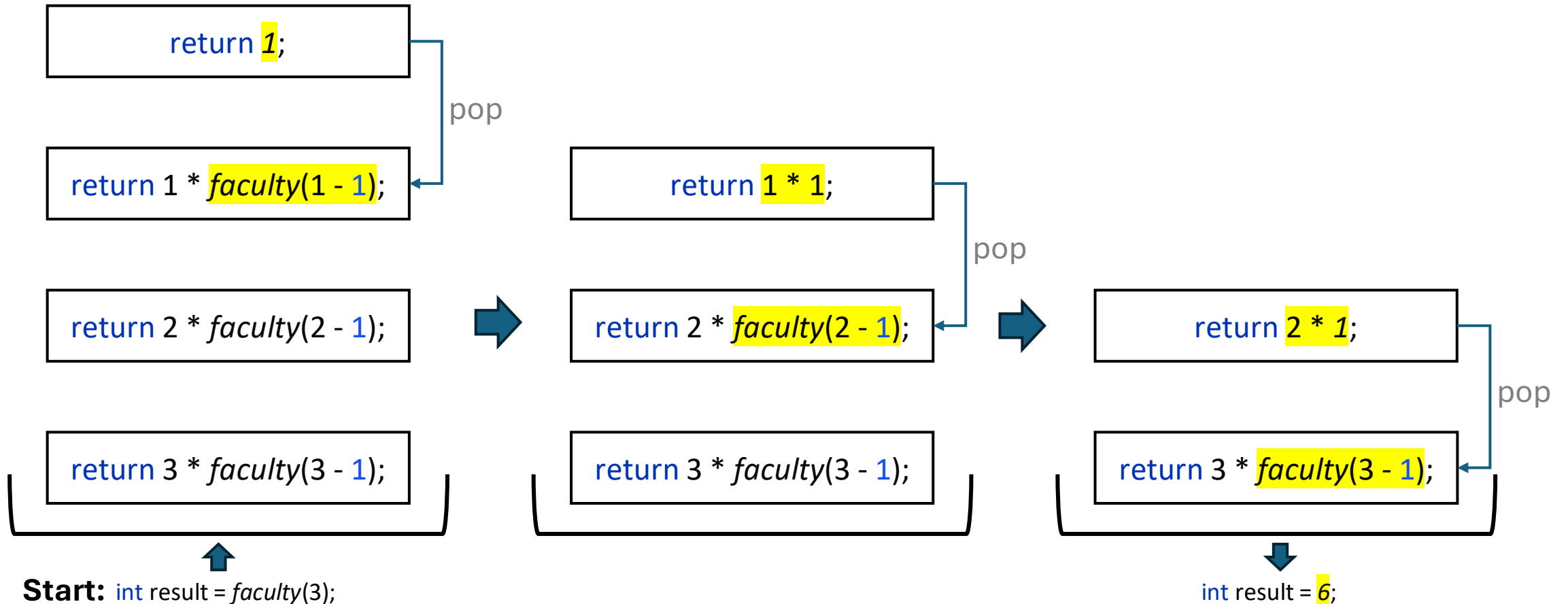
Parameter, lokale Variablen und Rückgabewerte werden im Stack gespeichert (push). Dies erledigt die Programmiersprache für uns automatisch.



# Stack: Rekursiver Abbau



Rekursive Aufrufe werden nacheinander ausgewertet und schrittweise vom Stack abgebaut (pop). Dies erledigt die Programmiersprache für uns automatisch.



# Der Arbeitsspeicher



Objekte werden im **Heap** gespeichert (**new**).

Alles andere wie lokale Variablen, Parameter, Rückgabewerte, primitive Variablen, Referenzen zu Objekten und Rücksprungadresse werden auf dem **Stack** gespeichert.

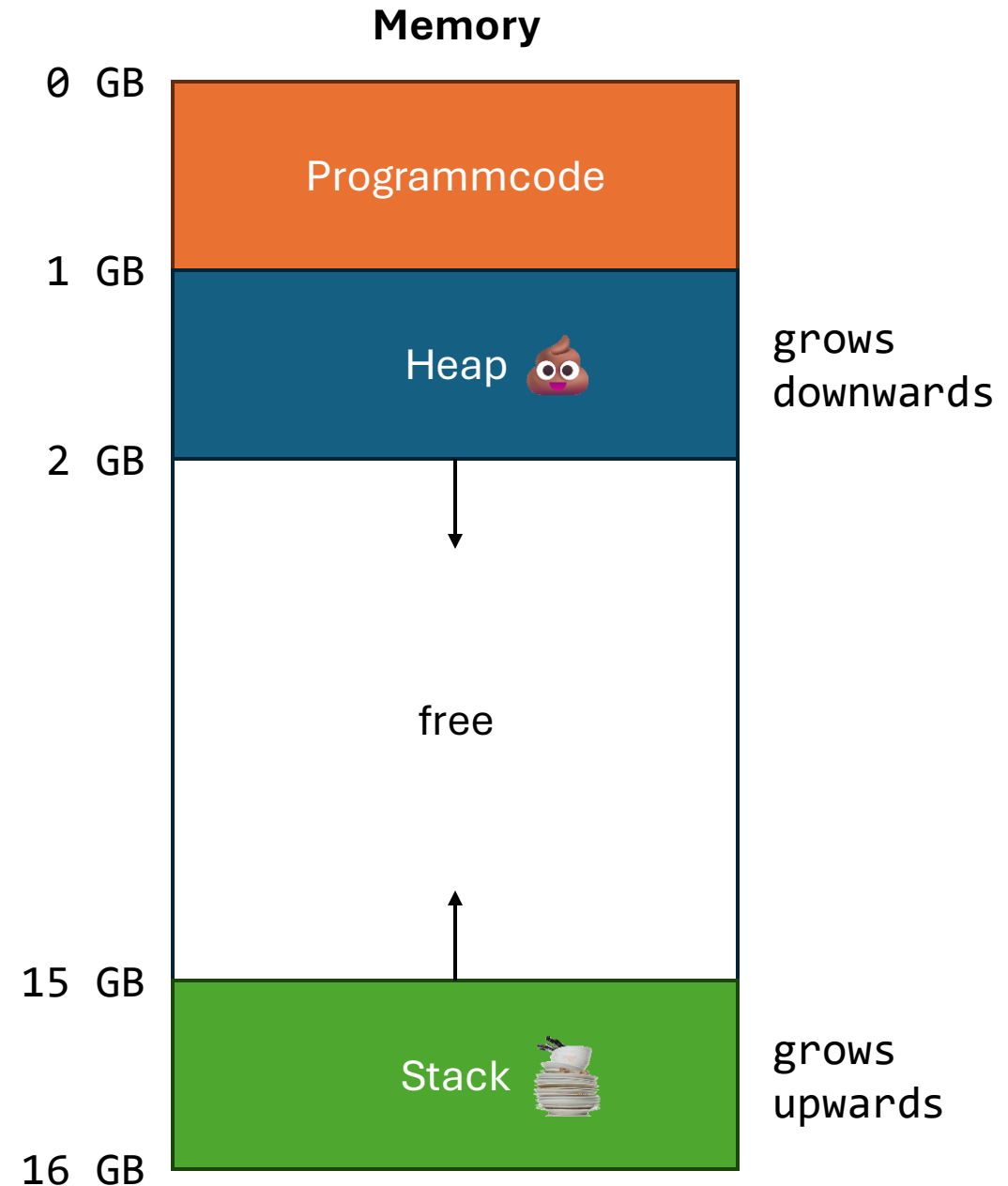
```
java -Xss128k -Xms512m -Xmx2048m myApp.java
```

**-Xss** setzt die maximale Größe des Stacks (pro Thread).

**-Xms** setzt die Anfangsgröße des Heaps (für die JVM).

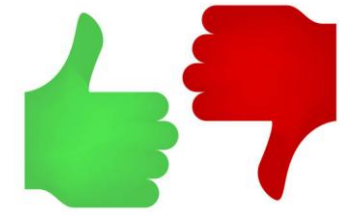
**-Xmx** setzt die maximale Größe des Heaps (für die JVM).

Siehe auch: [Java Command Line Options](#) oder `$ java -X`





# Rekursion: Vor- und Nachteile



## Vorteile:

- Rekursive Lösungen können **klarer und einfacher** sein **als ihre iterativen Lösungen**.  
Zum Beispiel beim **Traversieren** (Navigieren) von **Baumstrukturen** und **Graphen**.
- Rekursive Programme lassen sich häufig leicht **parallelisieren** (Multi-Threading).

## Nachteile:

- Rekursive Methoden haben in der Regel mehr **Overhead** als iterative Lösungen, da **jeder rekursive Aufruf Speicher auf dem Stack belegt**.
- Potenziell kann es zum **Stackoverflow** kommen, wenn kein freier Speicher mehr im Stack verfügbar ist.

# Rekursiv vs Iterativ

## Rekursiv

```
final class MathUtil {  
  
    static int faculty(int n) {  
        if (n == 0) {  
            return 1;  
        }  
        return n * faculty(n - 1);  
    }  
  
    void main() {  
        int n = 3;  
        int result = faculty(n);  
        IO.printf("%d ! = %d", n, result); // Output: 3! = 6  
    }  
}
```

## Iterativ

```
final class MathUtil {  
  
    static int facultyIterative(int n) {  
        int result = 1;  
        for (int i = 1; i <= n; i++) {  
            result *= i;  
        }  
        return result;  
    }  
  
    void main() {  
        int n = 3;  
        int result = facultyIterative(n);  
        IO.printf("%d ! = %d", n, result); // Output: 3! = 6  
    }  
}
```



Jede **Rekursion** lässt sich in eine **iterative** Lösung **umschreiben** und **umgekehrt**.

Zum Beispiel mit Hilfe von Java-Collections wie Deque (Stack) und Queue.

Das Umschreiben ist nicht immer so einfach wie in dem hier gezeigten Beispiel für die Fakultät.



# Endrekursion (Tail-Recursion)

**Endrekursion** (Tail-Recursion) ist eine spezielle Form der Rekursion, bei der der rekursive Aufruf der **letzte Schritt** in der Methode ist.

**Manche Rekursionen** lassen sich in eine **Endrekursion** umschreiben.

Die Endrekursion ermöglicht eine **Optimierung** durch den **Compiler** und **reduziert den Speicherverbrauch**, da immer nur der letzte Methodenaufruf auf dem Stack gespeichert werden muss.

Die Optimierung ist allerdings abhängig von der Programmiersprache. Tail-Recursion Optimization ist ein typisches Feature von funktionalen Programmiersprachen – Java hat aktuell **keine Optimierung** für Endrekursion.

# Endrekursion (Tail-Recursion): Beispiel

```
final class MathUtil {  
  
    static int faculty(int n) {  
        // Abort condition  
        if (n == 0) {  
            return 1;  
        }  
        // Recursive loop  
        return n * faculty(n - 1);  
    }  
  
    void main() {  
        int n = 3;  
        int result = faculty(n);  
        IO.printf("%d ! = %d", n, result); // Output: 3! = 6  
    }  
}
```

Handelt es sich hier um eine  
Endrekursion?



# Endrekursion: Beispiel

## Keine Tail-Recursion

```
final class MathUtil {  
  
    static int faculty(int n) {  
        // Abort condition  
        if (n == 0) {  
            return 1;  
        }  
        // No tail-recursion, because multiplication is the last action  
        return n * faculty(n - 1);  
    }  
  
    void main() {  
        int n = 3;  
        int result = faculty(n);  
        IO.printf("%d != %d", n, result); // Output: 3! = 6  
    }  
}
```

## Tail-Recursion

```
final class MathUtil {  
  
    static int faculty(int n) {  
        return facultyTail(n, 1); // result starts with the value 1  
    }  
  
    private static int facultyTail(int n, int result) {  
        // Abort condition  
        if (n == 0) {  
            return result;  
        }  
        // Tail recursion, because facultyTail is the last action  
        return facultyTail(n - 1, n * result); // pass result to next call  
    }  
  
    void main() {  
        int n = 3;  
        int result = faculty(n);  
        IO.printf("%d != %d", n, result); // Output: 3! = 6  
    }  
}
```



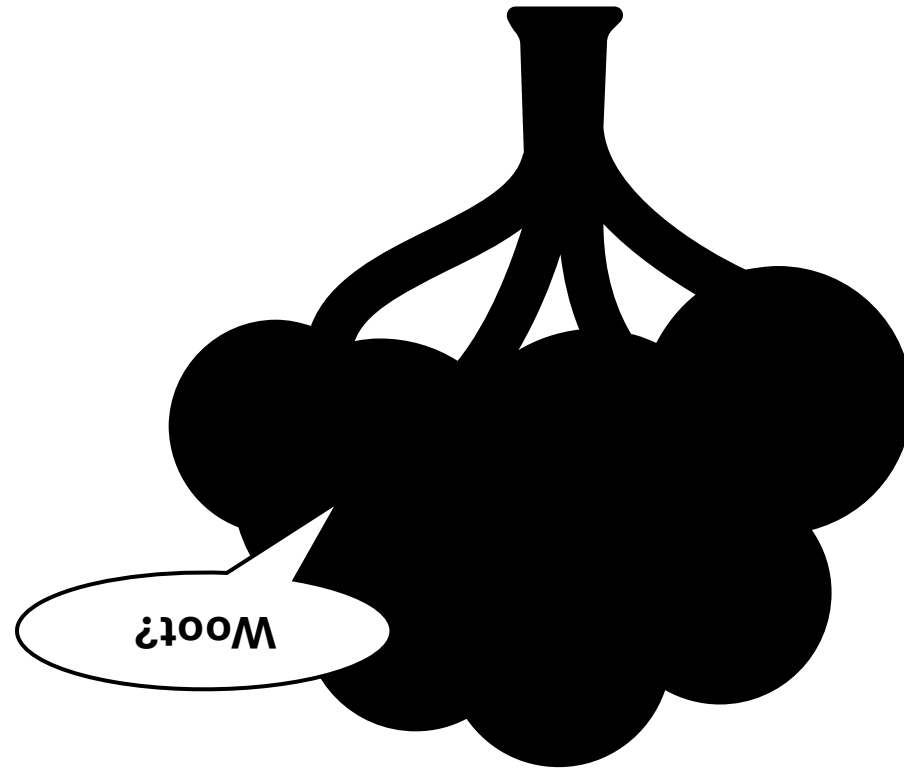
# Rekursion in der Praxis

Rekursion wird häufig in **funktionalen Programmiersprachen bevorzugt** (z. B. Haskell, Erlang, Elixir, Lisp).

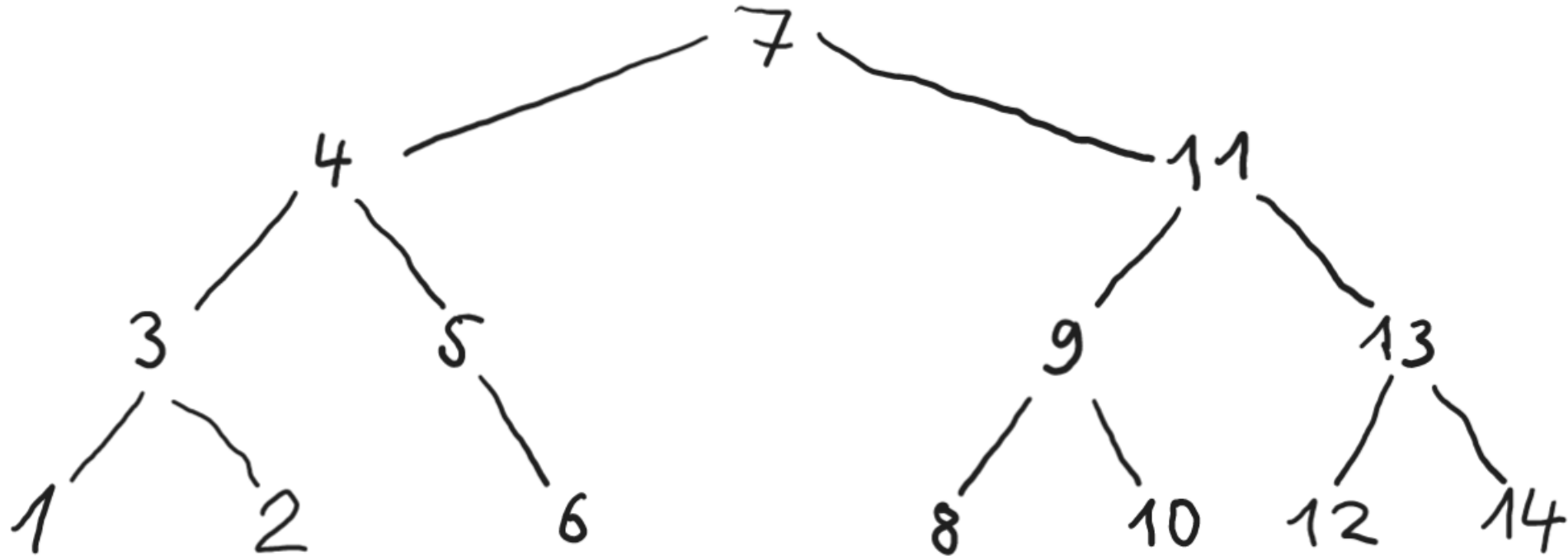
Weitere Anwendungsbeispiele für Rekursion ist z. B. das **Traversieren** (Navigieren) von **Graphen** und **Baumstrukturen** sowie **mathematische Funktionen, Sortialgorithmen** (z. B. Quicksort und Mergesort) und **Fraktale**.



# Baumstrukturen

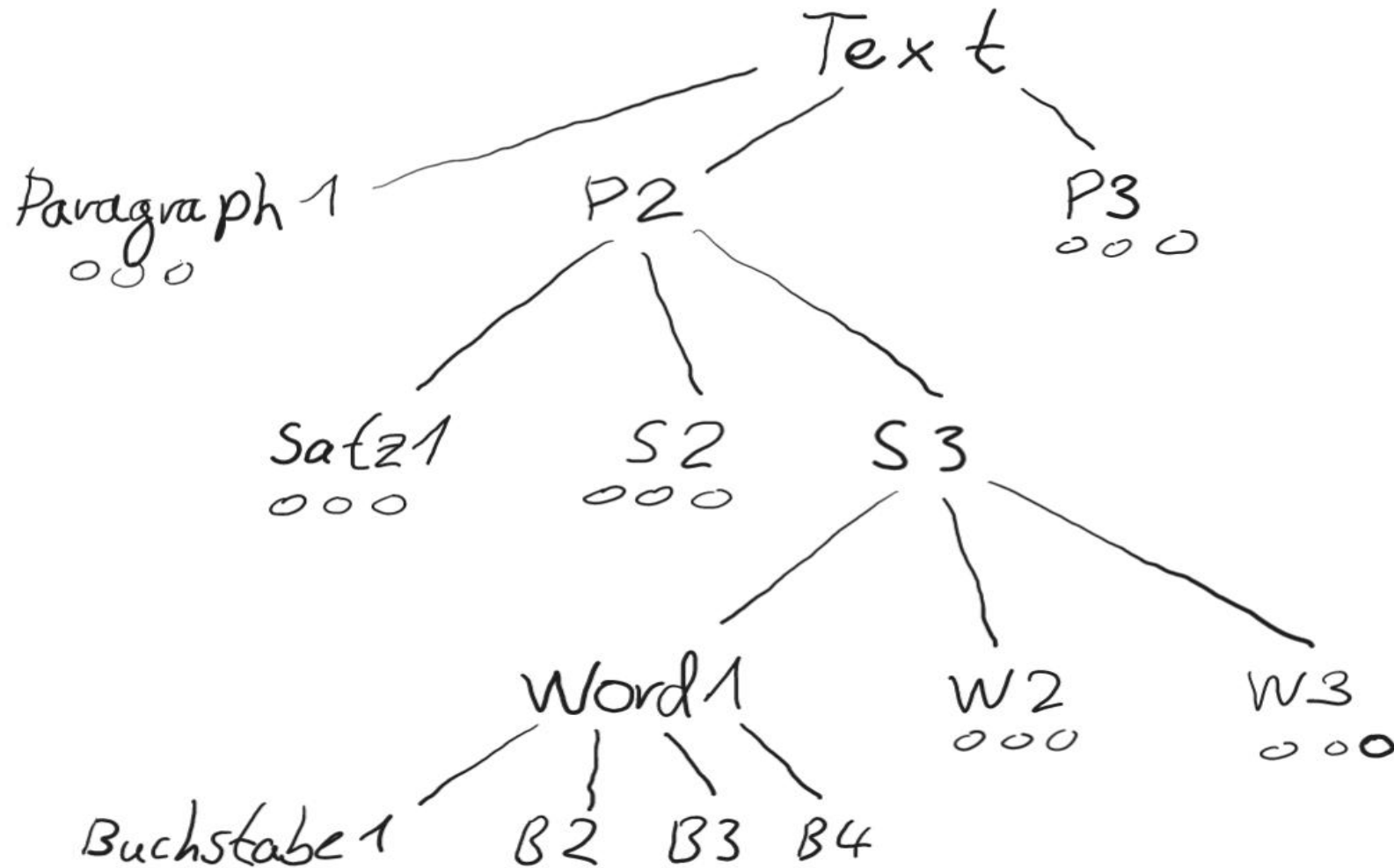


# Baumstrukturen: Binärbaum



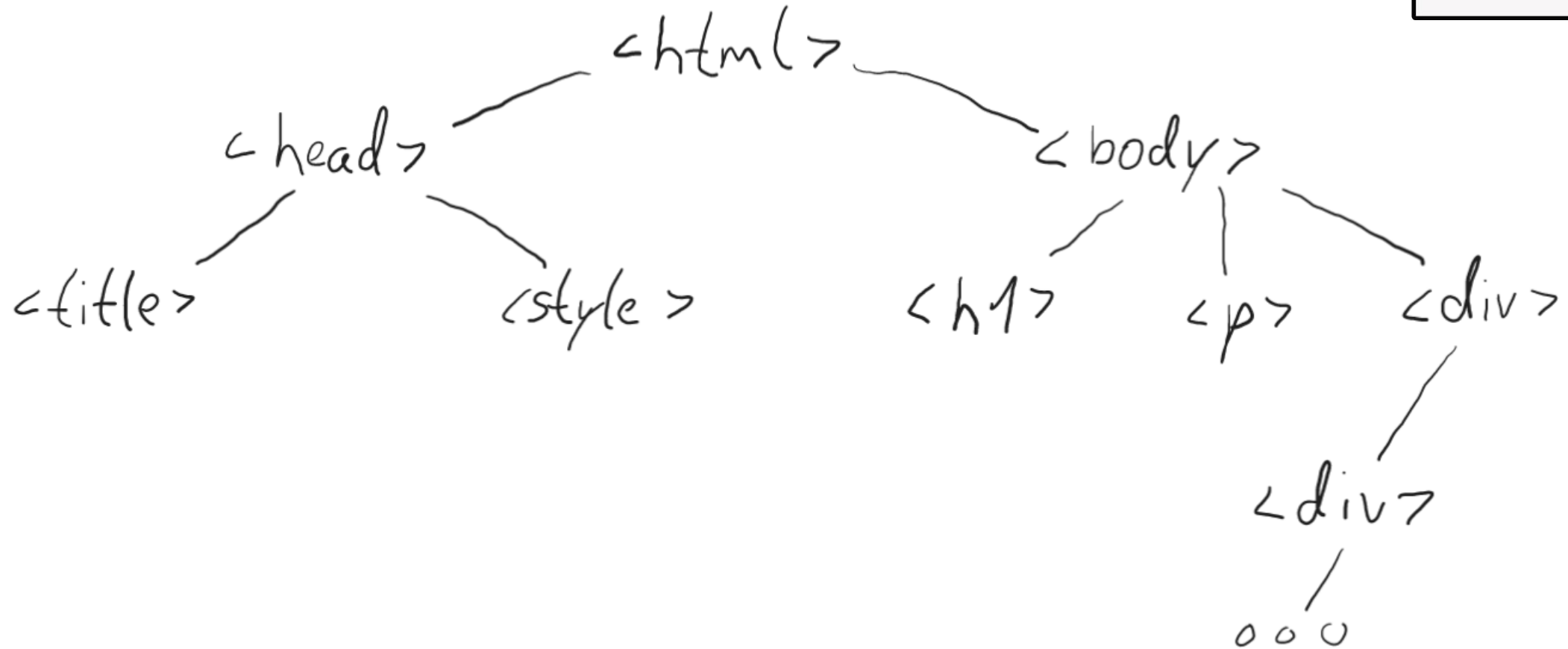


# Baumstrukturen: Sprache

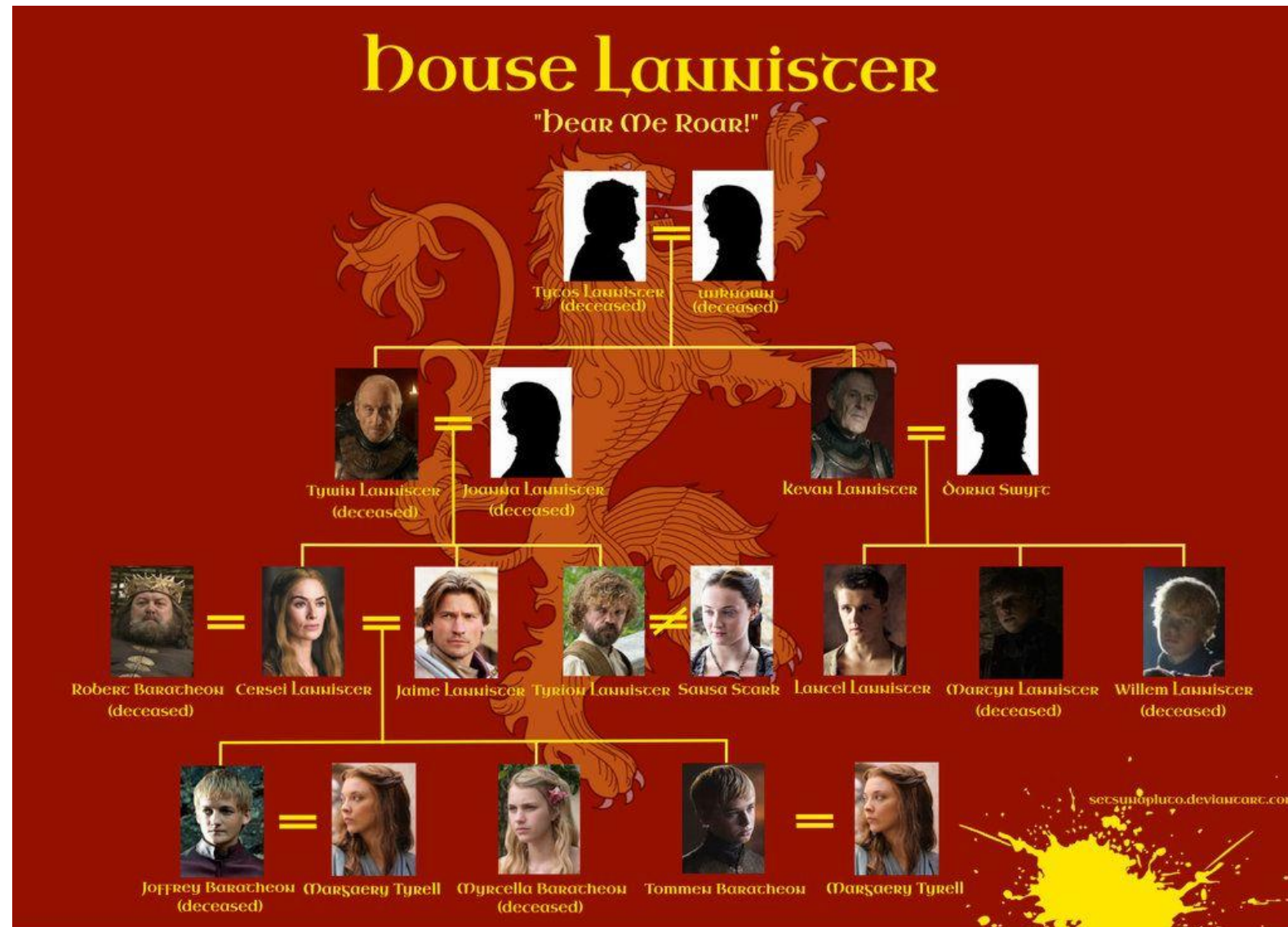


# Baumstrukturen: HTML

```
<html>
  <head>
    intelligent
  </head>
  <body>
    beautiful
  </body>
</html>
```



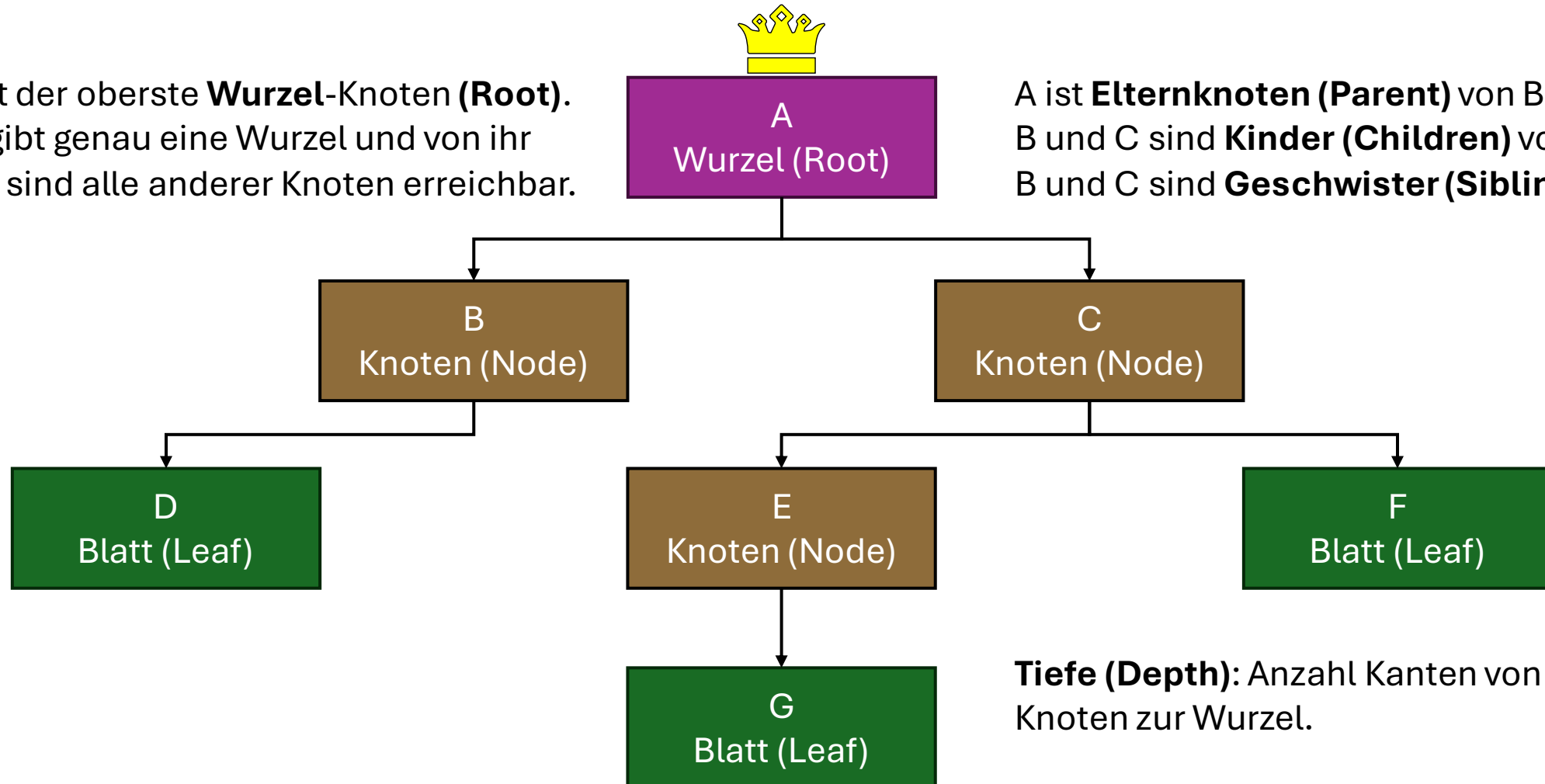
# Baumstrukturen: Stammbaum



# Baumstrukturen: Begriffe

Ein Baum besteht aus **Knoten (Nodes)**, die durch **Kanten (Edges)** miteinander verbunden sind.

A ist der oberste **Wurzel-Knoten (Root)**.  
Es gibt genau eine Wurzel und von ihr aus sind alle anderen Knoten erreichbar.



A ist **Elternknoten (Parent)** von B und C.  
B und C sind **Kinder (Children)** von A.  
B und C sind **Geschwister (Siblings)**.

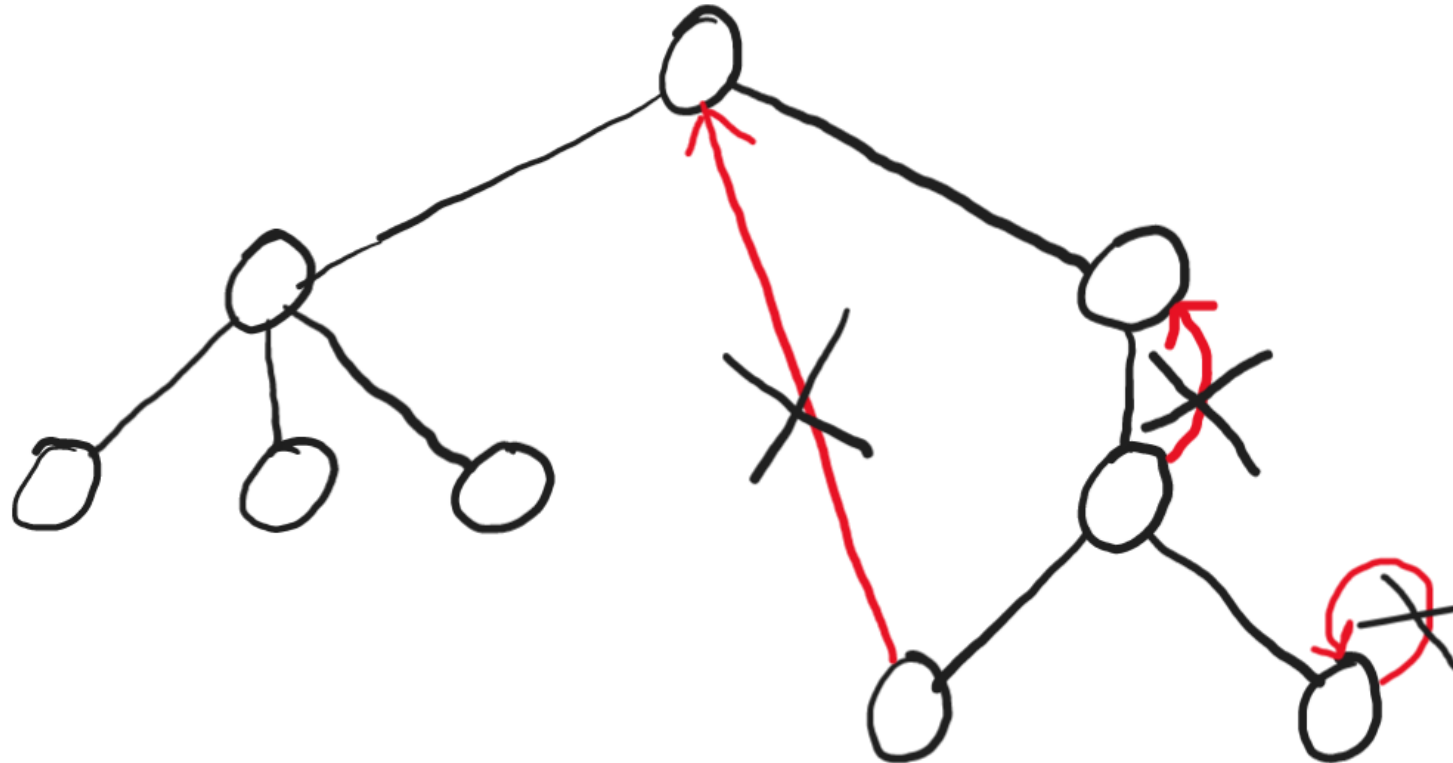
**Tiefe (Depth):** Anzahl Kanten von einem Knoten zur Wurzel.

**Höhe (Height):** Maximale Tiefe eines Knotens.

# Baumstrukturen: Azyklisch

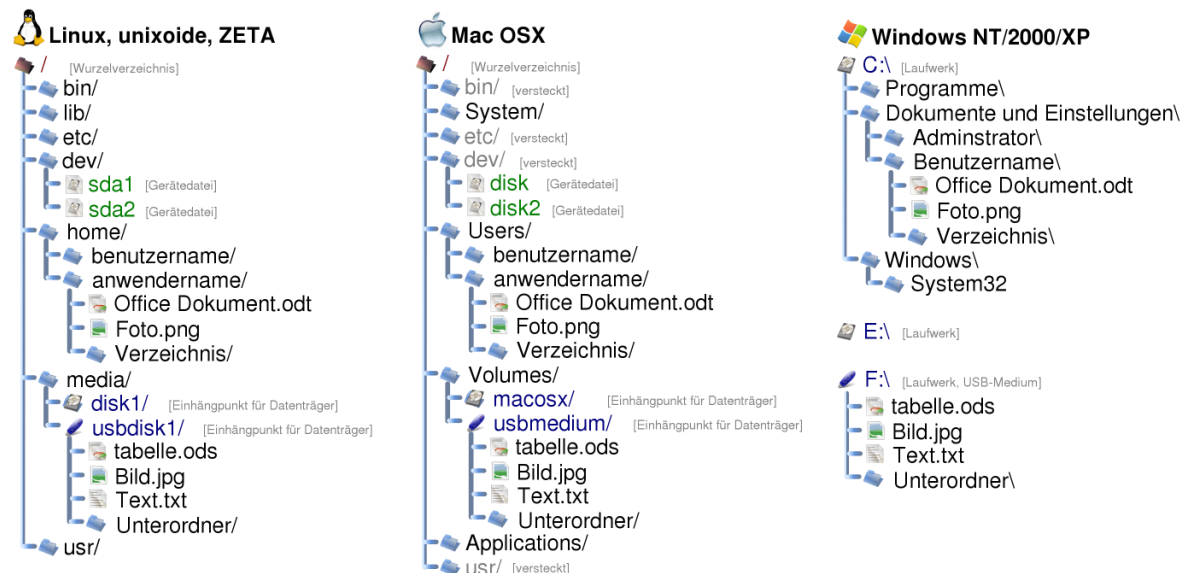


Baumstrukturen sind **hierarchisch** und **azyklisch**, sie enthalten keine rückläufigen Kanten bzw. Zyklen (im Gegensatz zu Graphen).

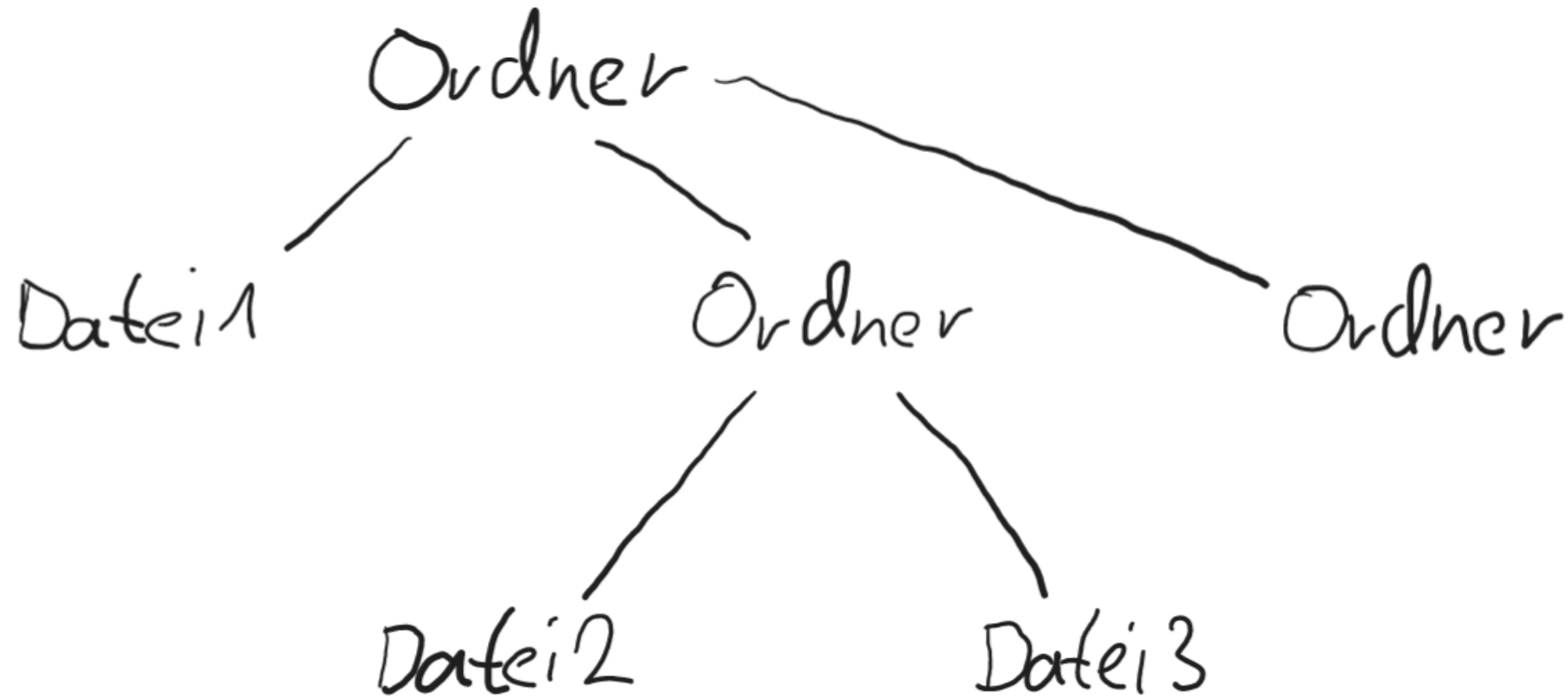


# Verzeichnisstruktur: Definition

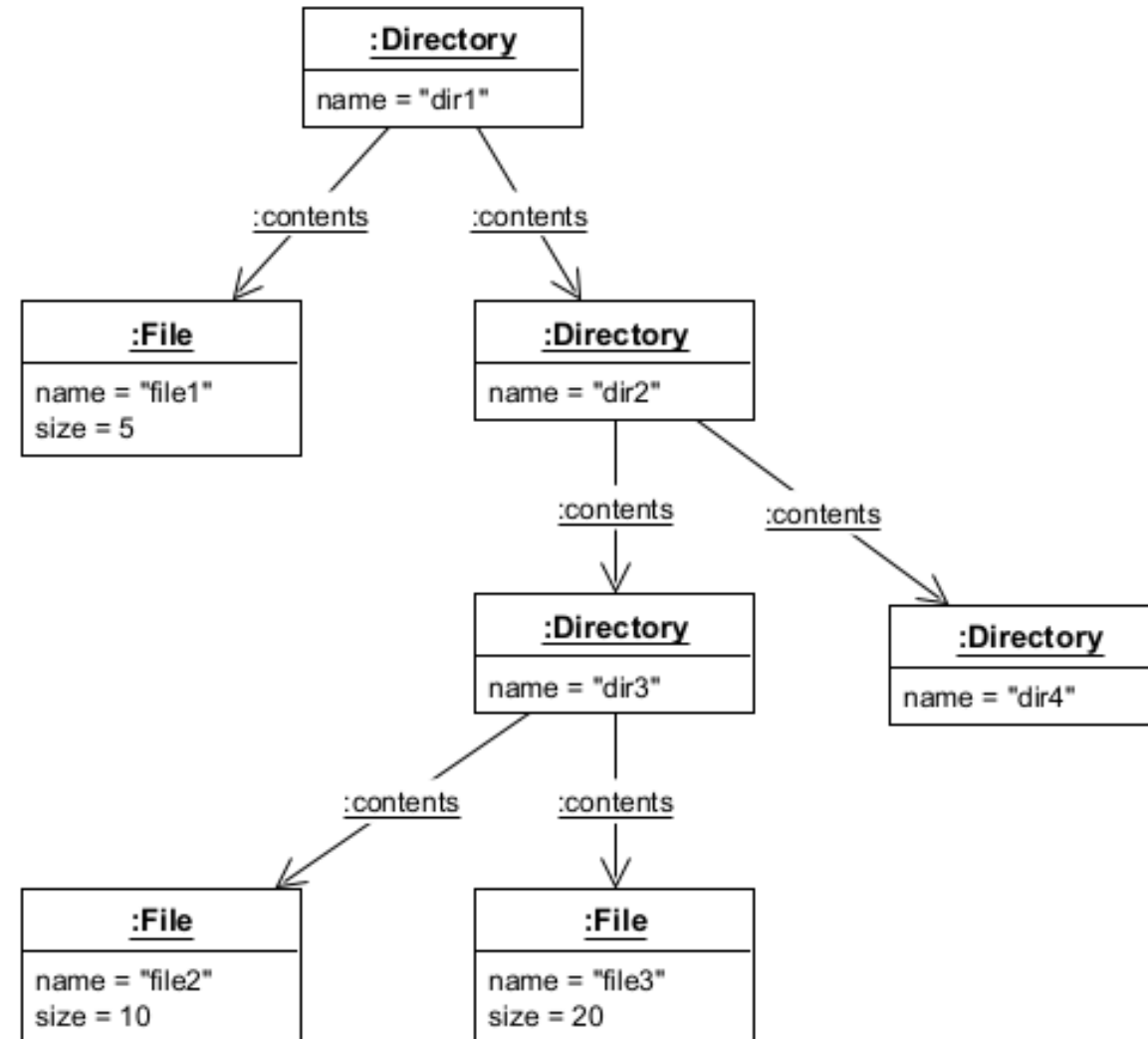
"Als Verzeichnisstruktur (auch Verzeichnisbaum oder Ordnerstruktur) wird im engeren Sinn die hierarchische Gestalt des gesamten Dateisystems eines einzelnen Computers bezeichnet und im weiteren Sinn ein Verzeichnisdienst für beliebige Objekte (wie z. B. Benutzer, Geräte, Dienste, Dateifreigaben und Pakete) eines Firmennetzes. Üblich ist eine **Baumstruktur**, die bei einer **Wurzel** beginnt und sich dann **beliebig verzweigt**." – [Wikipedia](#)



# Verzeichnisstruktur: Skizze



# Verzeichnisstruktur: Objektdiagramm

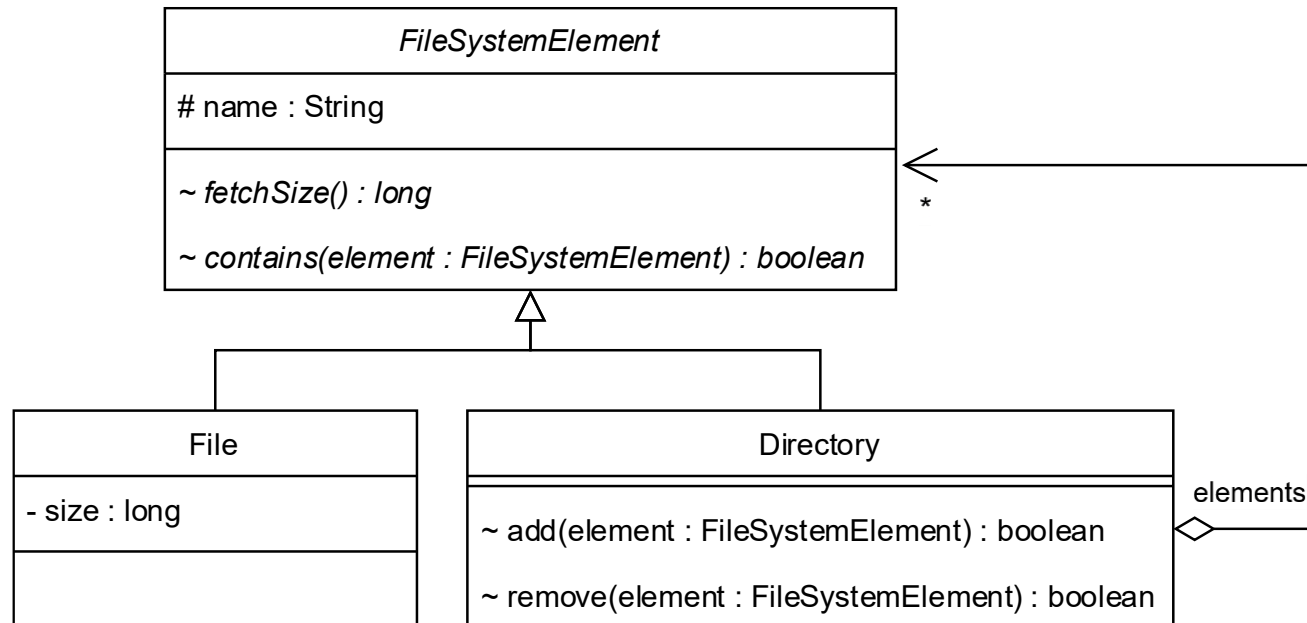
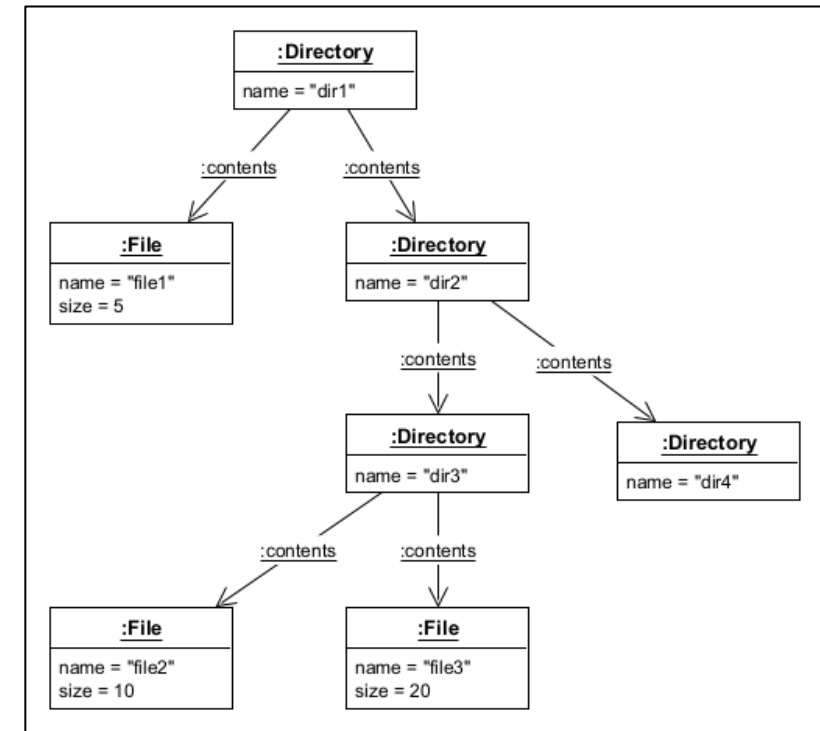




# Verzeichnisstruktur: Klassendiagramm



## Objektdiagramm

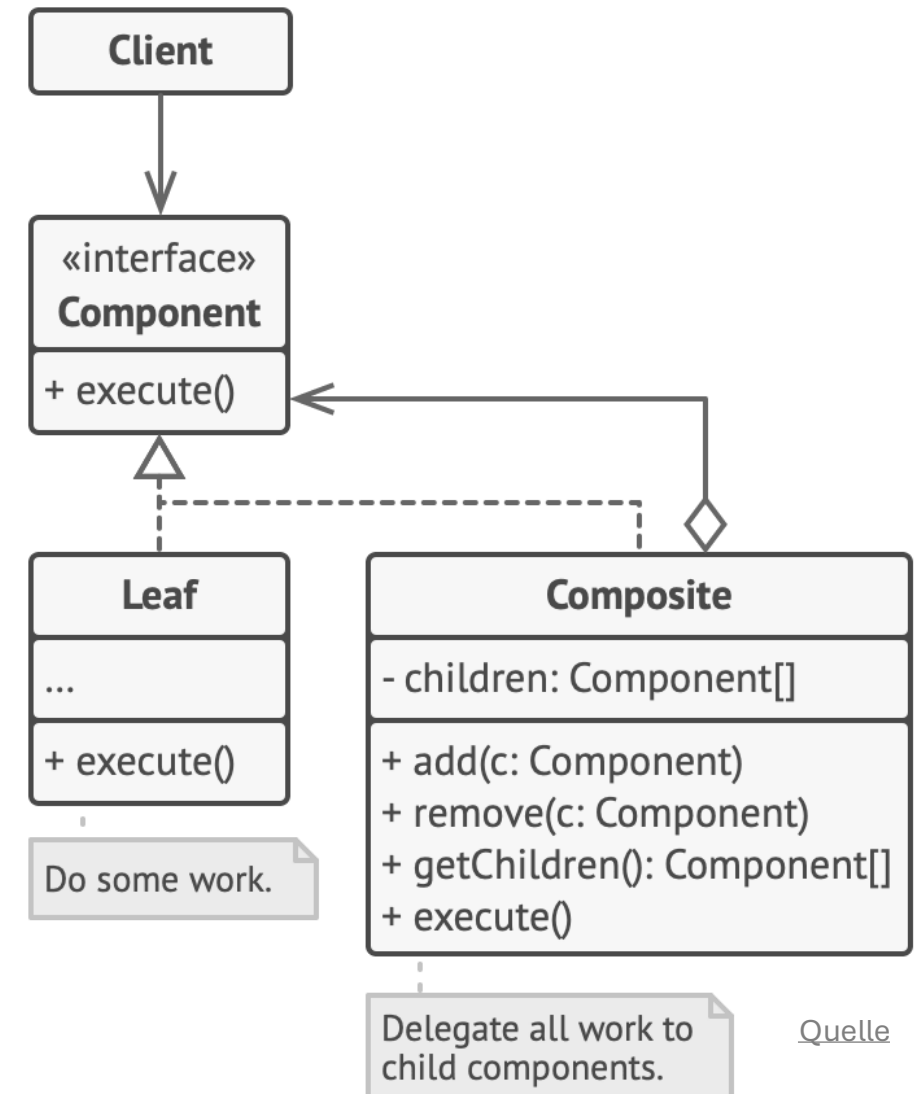


# Composite Pattern

Das Kompositum-Entwurfsmuster eignet sich für hierarchische Baum-Strukturen.

Atomare Komponenten (Leaf) und komplexe Komponenten (Node bzw. hier Composite) verwenden dieselbe Schnittstelle (Component).

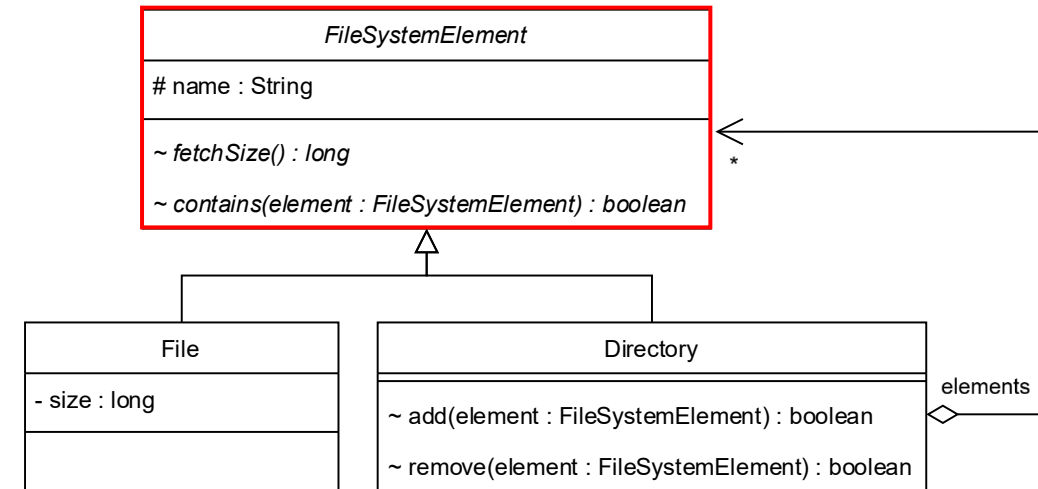
Component enthält typischerweise Methoden, die rekursiv durch die Komponenten traversieren, bis die atomaren Komponenten (Leafs) erreicht werden (Rekursionsabbruch).



Quelle

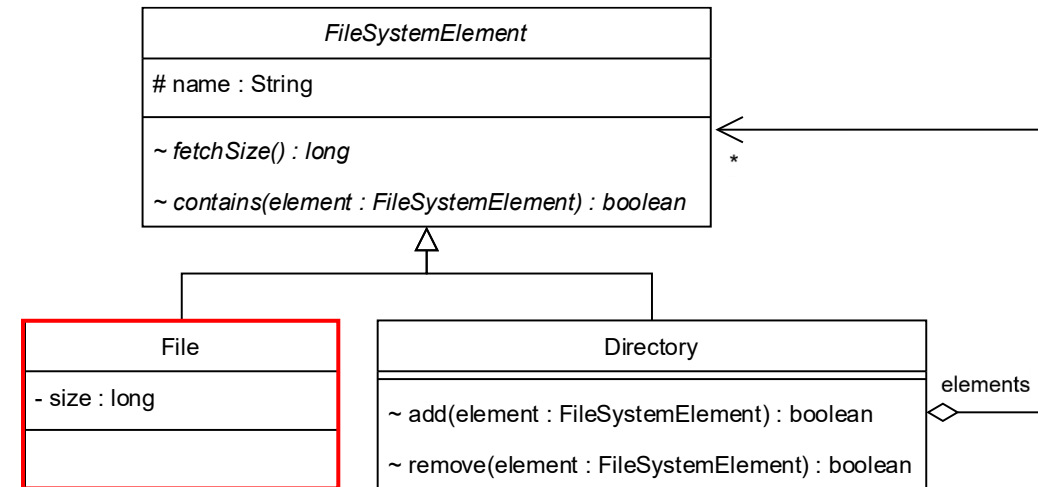
# FileSystemElement (Component)

```
abstract class FileSystemElement {  
  
    protected String name;  
  
    FileSystemElement(String name) {  
        this.name = name;  
    }  
  
    abstract long fetchSize();  
  
    abstract boolean contains(FileSystemElement element);  
  
    @Override  
    public String toString() {  
        return name;  
    }  
}
```



# File (Leaf)

```
class File extends FileSystemElement {  
  
    private final long size;  
  
    File(String name, long size) {  
        super(name);  
        this.size = size;  
    }  
  
    @Override  
    long fetchSize() {  
        return size;  
    }  
  
    @Override  
    boolean contains(FileSystemElement element) {  
        return false;  
    }  
}
```



# Directory (Composite)

```
class Directory extends FileSystemElement {  
  
    private final Collection<FileSystemElement> elements = new LinkedList<>();  
  
    Directory(String name) { super(name); }  
  
    boolean add(FileSystemElement element) { return elements.add(element); }  
    boolean remove(FileSystemElement element) { return elements.remove(element); }  
  
    @Override  
    long fetchSize() {  
        long result = 0;  
        for (FileSystemElement childElement : elements)  
            result += childElement.fetchSize();  
        return result;  
    }  
  
    @Override  
    boolean contains(FileSystemElement element) {  
        for (FileSystemElement childElement : elements)  
            if (childElement.equals(element) || childElement.contains(element)) // Tiefensuche.  
                return true;  
        return false;  
    }  
}
```

